Pokročilé funkce v SQL

Jiří Mareš

DATA academy



Co nás čeká

- Několik řetězcových funkcí
- Partitioning
- Úvod do datového slovníku
- Funkce PIVOT a UNPIVOT
- Funkce ROLLUP, CUBE a GROUPING
- Regulární výrazy
- Hierarchické dotazy
- Analytické funkce

CONVERT

Funkce slouží pro převod z jedné znakové sady do druhé.

Syntaxe:

convert(<řetězec>, <cílová_znaková_sada>[, <zdrojová_znaková_sada>])

Zdrojová sada je nepovinný parametr. Možné hodnoty, které lze jako parametr pro znakovou sadu použít, je možné najít níže uvedeným dotazem.

```
select
  *
from
  v$nls_valid_values
where
  parameter = 'CHARACTERSET';
```

CONVERT *(01_asql_convert.txt)*

Funkce je např. vhodná pro odstranění diakritiky.

Příklad:

Vypište jména všech klientů ze SAPu bez diakritiky.

```
select
 pt_party_name as jmeno,
 convert(pt_party_name, 'US7ASCII') as orezane_jmeno
from
 dwh_owner.parties
where
 ptori_key = 11;
```

JMENO	OREZANE_JMENO
Aarons Brett Scott	Aarons Brett Scott
Abdallahová Marie	Abdallahova Marie
Aberle Martin	Aberle Martin
Abraham Marek	Abraham Marek
Abrahamová Pavla	Abrahamova Pavla
Abrahamová Simona	Abrahamova Simona
Abrahamová Simona	Abrahamova Simona
Abrahamová Věra	Abrahamova Vera
Abrahám Vít	Abraham Vit
Absolon Miroslav	Absolon Miroslav
Absolonová Jiřina	Absolonova Jirina
Ackermann Emil	Ackermann Emil
Ackermann Emil	Ackermann Emil
Adam Jiří	Adam Jiri
Adam Miloš	Adam Milos

TRANSLATE

Funkce nám umožňuje v řetězci zaměnit jednu abecedu (=vybrané znaky) za abecedu jinou.

Syntaxe:

translate(<řetězec>, <zdrojová_abeceda>, <cílová_abeceda>).

Poznámka:

Obě abecedy by měly být ideálně stejně dlouhé. Vyhodnocení pak probíhá tak, že první znak zdroje se zamění za cíl, poté druhý za druhý atd.

Pokud je vstupní abeceda delší o n znaků, pak je posledních n znaků zaměněno za null. Je-li naopak delší cílová abeceda, pak se nic neděje, tyto nadbytečné znaky jsou ignorovány.

TRANSLATE *(02_asql_translate.txt)*

Příklad:

Vypište jména všech klientů ze SAPu bez diakritiky.

```
select
  pt_party_name as jmeno,
  translate
    (
      pt_party_name,
      'aábcčdďeéěfghiíjklmnňoópqrřsštťuúůvwxyzžAÁBCČDĎEÉĚFGHIÍJKLMNŇOÓPQRŘSŠTŤUÚŮVWXYZŽ',
      'aabccddeeefghiijklmnnoopqrrssttuuuvwxyzzAABCCDDEEEFGHIIJKLMNNOOPQRRSSTTUUUVWXYZZ'
    ) as orezane_jmeno
from
  dwh_owner.parties
where
  ptori_key = 11;
```

Poznámka:

V tomto případě by mi ale další znaky (jako ö, ü, Í apod.) zůstaly nezměněny.

TRANSLATE *(03_asql_zadani_01.txt)*

Úkol:

Zkuste rozluštit text IXYLOLJKFXSFKZFQFJMOLYRPROMRQKXMOXZBSPBWJRWBSBODFIFRP zašifrovaný pomocí Caesarovy šifry. Šifrovací abeceda je anglická, tedy 26 znaků bez diakritiky od A do Z.

Řešení:

```
select
  translate
(
'IXYLOLJKFXSFKZFQFJMOLYRPROMRQKXMOXZBSPBWJRWBSBODFIFRP',
'ABCDEFGHIJKLMNOPQRSTUVWXYZ',
'DEFGHIJKLMNOPQRSTUVWXYZABC'
) as text
from
  dual;
```

- „Labor omnia vincit improbus."
  – *„Urputná práce vše zmůže."*
  Vergilius

TEXT
LABOROMNIAVINCITIMPROBUSURPUTNAPRACEVSEZMUZEVERGILIUS

LISTAGG

Agregační funkce, pomocí které dokážeme vypsat více hodnot na jeden řádek.

Syntaxe:

listagg(<sloupec>, <oddělovač>) within group (order by <řazení>)

Funkce vypíše všechny hodnoty <sloupce> oddělené znakem <oddělovač> a seřazené je sloupce / výrazu <řazení>.

LISTAGG *(04_asql_listagg_01.txt)*

Příklad:

Pro každý tým vypište abecedně všechny jeho zaměstnance.

```
select
  t.emptm_source_id,
  listagg(p.pt_party_name, ', ') within group (order by p.pt_party_name) as seznam
from
  dwh_owner.employees e,
  dwh_owner.employee_teams t,
  dwh_owner.parties p
where
  e.emptm_key = t.emptm_key and
  e.pt_key = p.pt_key and
  e.ptori_key = p.ptori_key and
  p.ptori_key = 11 and
  p.pt_deleted_flag = 'N' and
  t.emptm_key > 0
group by
  t.emptm_source_id
order by
  t.emptm_source_id;
```

EMPTM_SOURCE_ID	SEZNAM
ANSCHL001	Chura Jan, Ctiborová Hana, Dostálová Nikola, Hodík Jan, Jansová Lenka, Kodera Lukáš, Kopecká Blanka, Krupičková Zuzana, Lév Vladimír, Novák Michal, Skokánek Luděk, Vejdovský Tomáš, Zdrůbecká Karin, Špaček Luděk
ANSCHL002	Dudáš Vojtěch, Kala Jan, Koželský Marek, Kvita Pavel, Machovec Libor, Ptáček Miloš, Černý Robert
ANSCHL003	Fejfárek Tomáš, Hošťáková Hana, Kunčar Miroslav, Zamborská Anna
ANSCHL004	Braun Michael, Marek Jakub
ANSDESCHL001	Holečková Denisa, Klimek David, Lupan Alexandru, Margaris Nikos, Vokr Michal, Zientková Jana
ANSDESCHL002	Badík Marek, Bednář Miloš, Fišerová Lucie, Hanzal Jiří, Pecklová Jarmila, Sippl Antonín, Skálová Zuzana, Sojka Martin, Tadial Lukáš, Veigertová Markéta, Šedivá Pavlína, Šimpachová Daniela, Šinágl Martin, Šuchmová Rosická Tereza
ANSDESCHL003	Antony Martin, Baník Ján, Draganov Vojtěch, Kadlecová Helena, Kopecký Martin, Leľo Zuzana, Lidmila Karel, Pergler Aleš, Procházka Pavel, Pšenička Jaroslav, Rafaj Andrej, Selčan Vladimír, Sojka Štěpán, Švec Petr
ANSDESCHL004	Frank Václav, Jelínek Jaroslav, Jesenský Marek, Mošničková Zuzana, Tauchmanová Kristýna, Šich Jan

LISTAGG *(05_asql_listagg_02.txt)*

Poznámka:

Pro správné řazení je nutné použít funkci nlssort s parametrem xczech.

```
select
  t.emptm_source_id,
  listagg(p.pt_party_name, ', ') within group (order by nlssort((p.pt_party_name),'nls_sort=xczech')) as seznam
from
  dwh_owner.employees e,
  dwh_owner.employee_teams t,
  dwh_owner.parties p
where
  e.emptm_key = t.emptm_key and
  e.pt_key = p.pt_key and
  e.ptori_key = p.ptori_key and
  p.ptori_key = 11 and
  p.pt_deleted_flag = 'N' and
  t.emptm_key > 0
group by
  t.emptm_source_id
order by
  t.emptm_source_id;
```

EMPTM_SOURCE_ID	SEZNAM
ANSCHL001	Ctiborová Hana, Dostálová Nikola, Hodík Jan, Chura Jan, Jansová Lenka, Kodera Lukáš, Kopecká Blanka, Krupičková Zuzana, Lév Vladimír, Novák Michal, Skokánek Luděk, Špaček Luděk, Vejdovský Tomáš, Zdrůbecká Karin
ANSCHL002	Černý Robert, Dudáš Vojtěch, Kala Jan, Koželský Marek, Kvita Pavel, Machovec Libor, Ptáček Miloš
ANSCHL003	Fejfárek Tomáš, Hošťáková Hana, Kunčar Miroslav, Zamborská Anna
ANSCHL004	Braun Michael, Marek Jakub
ANSDESCHL001	Holečková Denisa, Klimek David, Lupan Alexandru, Margaris Nikos, Vokr Michal, Zientková Jana
ANSDESCHL002	Badík Marek, Bednář Miloš, Fišerová Lucie, Hanzal Jiří, Pecklová Jarmila, Sippl Antonín, Skálová Zuzana, Sojka Martin, Šedivá Pavlína, Šimpachová Daniela, Šinágl Martin, Šuchmová Rosická Tereza, Tadial Lukáš, Veigertová Markéta
ANSDESCHL003	Antony Martin, Baník Ján, Draganov Vojtěch, Kadlecová Helena, Kopecký Martin, Leľo Zuzana, Lidmila Karel, Pergler Aleš, Procházka Pavel, Pšenička Jaroslav, Rafaj Andrej, Selčan Vladimír, Sojka Štěpán, Švec Petr
ANSDESCHL004	Frank Václav, Jelínek Jaroslav, Jesenský Marek, Mošničková Zuzana, Šich Jan, Tauchmanová Kristýna

LISTAGG

Úkol:

Pro každého nesmazaného unifikovaného klienta (fyzickou osobu) vypište všechny jeho platné identifikátory (party_ids) jako „hodnota (typ)“ na jeden řádek oddělené čárkou a setříděné dle typu a hodnoty. Vyberte pouze typy se zdrojem 'EX0'.

Řešení:

```
select
  p.pt_party_name,
  listagg(id.ptid_code || ' (' || tp.ptidtp_unified_id || ')', ', ') within group (order by tp.ptidtp_unified_id, id.ptid_code) as id
from
  dwh_owner.parties p,
  dwh_owner.party_ids id,
  dwh_owner.party_id_types tp
where
  p.pttp_key = 1 and
  p.ptori_key = 4 and
  p.pt_deleted_flag = 'N' and
  p.pt_key = id.pt_key and
  p.ptori_key = id.ptori_key and
  sysdate between id.ptid_valid_from and id.ptid_valid_to and
  id.ptidtp_key = tp.ptidtp_key and
  tp.ptidtp_source_system_id = 'EX0'
group by
  p.pt_key,
  p.pt_party_name
order by
  p.pt_party_name;
```

PT_PARTY_NAME	ID
A. Al-Dubaee Shawki	02166914 (CP), 201668720 (CP), 201668720 (OP)
A Almass Shaima Khalid	T890863 (CPC)
A Alomari Saud Zaid	000234121 (PDP)
A Alshehri Ahmed Ali	I1853777 (CPC)
A c Dinesh	08994794 (CPC)

Partitioning

Co je to?

Partitioning je technologie sloužící v relační databázi k fyzickému rozdělení rozsáhlých datových tabulek do menších částí na základě logického členění dat v tabulce, nazývaných partition (český přibližný ekvivalent oddíl se příliš neadaptoval).

Partitioning

Co je to?

V DWH využíváme u velkého množství tabulek, abychom omezili množství dat, se kterým se pracuje. Až na výjimky používáme dva typy:

LIST PARTITIONING

Oddíly jsou tvořené přesnou hodnotou klíče, nebo výčtem těchto hodnot. Typickým příklad v DWH jsou sloupce ptori_key, acctp_key a source_system_id.

RANGE PARTITIONING

Oddíly jsou tvořeny „časovým intervalem“. Typicky máme v DWH oddíly denní (transakce, zůstatky), měsíční (fakta) a speciální, často tříměsíční (historie, klasifikace).

V některých případech je využito i členění pomocí dvou klíčů – často kombinace typu účtu, klasifikace či klienta s datem – jedná se opět o range partitioning.

Partitioning

Použití

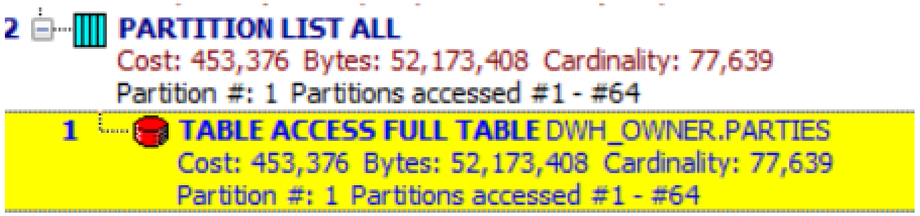
Pokud potřebujete pracovat jen s omezenou skupinou dat z nějaké tabulky, a tato je členěná na partitions, je třeba znát „partition“ klíče a v maximální míře je využívat. Jinak budeme načítat zbytečné množství dat, které nepotřebujeme.

Příklad:

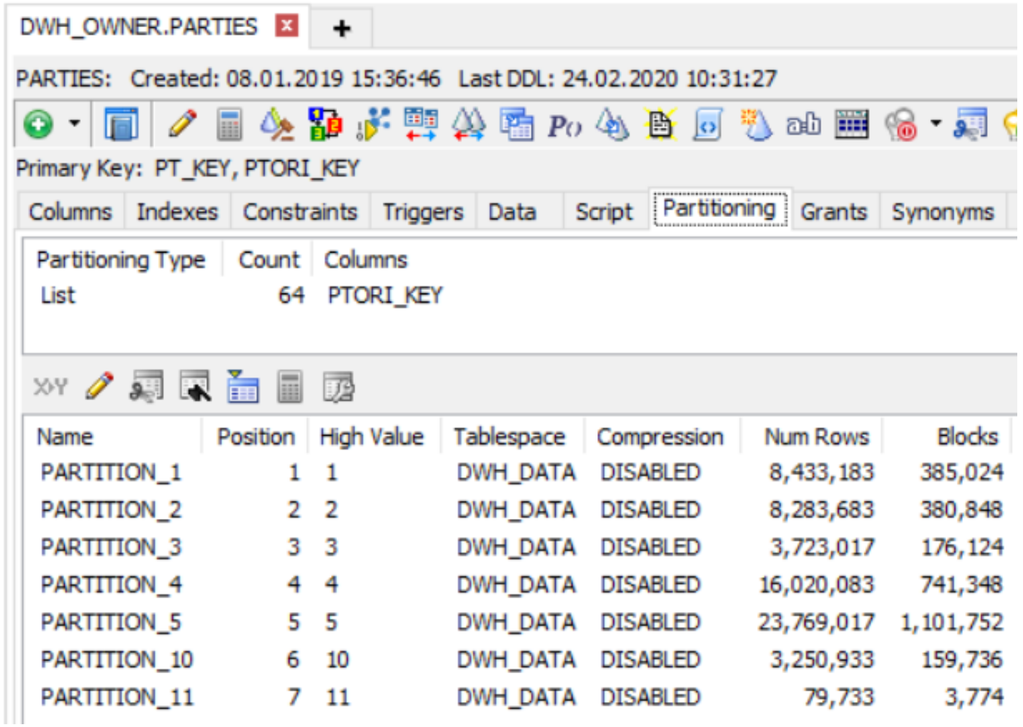
*select * from dwh_owner.parties where ptori_key = 11;*



*select * from dwh_owner.parties where pt_source_system_id = 'HR0';*



Partitioning



Poznámka:

*select * from dwh_owner.parties where ptori_key = 11;*

Můžeme přepsat použitím klíčového slova **PARTITION (<jméno partition>).**

*select * from dwh_owner.parties **partition (partition_11)**;*

Datový slovník

Co je to?

Datový slovník („data dictionary“) je soubor tabulek a pohledů, který nám umožňuje nahlédnout do vnitřního fungování a struktur databáze ORACLE.

My si představíme tzv. statické pohledy.

Většina pohledů existuje ve třech variantách, s prefixy USER, ALL a DBA.

USER – v tomto typu pohledů uvidíte jen informace, které se týkají vámi vlastněných objektů

ALL – v tomto typu pohledů uvidíte jen informace, které se týkají objektů, na něž máte práva

DBA – v tomto typu pohledů uvidíte o všech objektech.

Poznámka:

K USER a ALL pohledům mají práva všichni uživatelé, na DBA pohledy je třeba speciální oprávnění.

Datový slovník

Základní pohledy:

Název pohledu	Význam
ALL_OBJECTS	Seznam všech databázových objektů
ALL_TABLES	Seznam všech tabulek
ALL_VIEWS	Seznam všech pohledů
ALL_INDEXES	Seznam všech indexů
ALL_CONSTRAINTS	Seznam „omezení“ – primární klíč (P), unikátní klíč (U), cizí klíč (R) a kontrola (C).
ALL_TRIGGERS	Seznam všech triggerů

Tabulkové pohledy:

Název pohledu	Význam
ALL_TAB_COLUMNS	Seznam všech sloupců tabulek
ALL_TAB_COMMENTS	Seznam komentářů k tabulkám
ALL_TAB_HISTOGRAMS	Seznam všech histogramů
ALL_TAB_PARTITIONS	Seznam všech partition tabulek
ALL_IND_PARTITIONS	Seznam všech parttiion indexů

Datový slovník

„Partition“ pohledy:

Název pohledu	Význam
ALL_PART_TABLES	Seznam všech „partyšnovaných“ tabulek
ALL_PART_INDEXES	Seznam všech „partyšnovaných“ indexů
ALL_PART_KEY_COLUMNS	Seznam všech „partyšnovacích“ klíčů tabulek a indexů
ALL_PART_COL_STATISTICS	Seznam všech statistik nad „partyšnovanými“ sloupci
ALL_PART_HISTOGRAMS	Seznam všech statistik nad „partyšnovanými“ sloupci

Datový slovník

Úkol:

Vypište nejvyšší počet partition, které má nějaká tabulka ve schématu DWH_OWNER.

Řešení:

```
select
  max(pocet) as pocet
from
  (
    select
      count(1) as pocet
    from
      all_tab_partitions
    where
      table_owner = 'DWH_OWNER'
    group by
      table_name
  )
;
```

Datový slovník

Úkol:

Porovnejte výsledek s dotazem níže.

```
select
  max(partition_count) as pocet
from
  all_part_tables
where
  owner = 'DWH_OWNER'
;
```

Typ partitioningu pomocí intervalu vrací výchozí hodnotu, namísto skutečného počtu již vytvořených partitions.

Datový slovník

Úkol:

Vypište název sloupce, jehož jméno je ve schématu DWH_OWNER nejčetnější. Nepoužívejte analytické funkce!

Řešení:

```
select
  column_name
from
  all_tab_columns
where
  owner = 'DWH_OWNER'
group by
  column_name
having count(1)
  =
    (
      select max(pct) from (select count(1) as pct from all_tab_columns where owner = 'DWH_OWNER' group by column_name)
    );
```

Datový slovník

Úkol:

Vypište ty tabulky ze schématu DWH_OWNER, které mají více než jeden partition sloupec.

Řešení:

```
select
  name
from
  all_part_key_columns
where
  owner = 'DWH_OWNER' and
  object_type = 'TABLE'
group by
  name
having
  count(1) > 1
order by
  name;
```

```
select
  table_name
from
  all_part_tables
where
  owner = 'DWH_OWNER' and
  partitioning_key_count > 1
order by
  table_name
;
```

PIVOT

Funkce pivot slouží k tomu, abychom mohli několik řádků zobrazit jako sloupce.

Například zůstatky ke konci měsíce máme v tabulce account_balances. Pokud bychom nás zajímalo posledních šest měsíců, dotaz nám vrátí šest řádku. Pomocí funkce pivot můžeme výsledek „překlopit“ do jediného řádku.

Syntaxe:

```
select
*
from
(
  <vnitřní select>
)
pivot (<agregační funkce> for <metrika> in (<výčet hodnot>));
```

Jelikož je obecný zápis velmi neintuitivní, rovnou si ho ukážeme na příkladu.

PIVOT *(06_asql_pivot.txt)*

Příklad:

Pro druhé pololetí roku 2025 vypište zůstatky ke konci měsíců pro běžné účty ze Symbols.

Řešení:

```
select
  acc_key,
  prosinec_balance, listopad_balance, rijen_balance, zari_balance, srpen_balance, cervenec_balance
from
  (
    select
      acc_key,
      accfc_balance_czk,
      accfc_period
    from
      dwh_owner.current_account_facts
    where
      acctp_key = 602 and
      accfc_period in
      (
        to_date('20251231', 'YYYYMMDD'), to_date('20251130', 'YYYYMMDD'), to_date('20251031', 'YYYYMMDD'), to_date('20250930', 'YYYYMMDD'),
        to_date('20250831', 'YYYYMMDD'), to_date('20250731', 'YYYYMMDD')
      )
  )
pivot
  (
    max(accfc_balance_czk) as balance for accfc_period in (to_date('20251231', 'YYYYMMDD') as prosinec, to_date('20251130', 'YYYYMMDD') as listopad,
to_date('20251031', 'YYYYMMDD') as rijen, to_date('20250930', 'YYYYMMDD') as zari, to_date('20250831', 'YYYYMMDD') as srpen, to_date('20250731', 'YYYYMMDD') as
cervenec)
  );
```

PIVOT

Stejný výstup bychom dokázali udělat i pomocí funkce case.

Řešení:

```
select
  acc_key,
  max(case when accfc_period = to_date('20251231', 'YYYYMMDD') then accfc_balance_czk else 0 end) as prosinec_balance,
  max(case when accfc_period = to_date('20251130', 'YYYYMMDD') then accfc_balance_czk else 0 end) as listopad_balance,
  max(case when accfc_period = to_date('20251031', 'YYYYMMDD') then accfc_balance_czk else 0 end) as rijen_balance,
  max(case when accfc_period = to_date('20250930', 'YYYYMMDD') then accfc_balance_czk else 0 end) as zari_balance,
  max(case when accfc_period = to_date('20250831', 'YYYYMMDD') then accfc_balance_czk else 0 end) as srpen_balance,
  max(case when accfc_period = to_date('20250731', 'YYYYMMDD') then accfc_balance_czk else 0 end) as cervenec_balance
from
  dwh_owner.current_account_facts
where
  acctp_key = 602 and
  accfc_period in
  (
    to_date('20251231', 'YYYYMMDD'),
    to_date('20251130', 'YYYYMMDD'),
    to_date('20251031', 'YYYYMMDD'),
    to_date('20250930', 'YYYYMMDD'),
    to_date('20250831', 'YYYYMMDD'),
    to_date('20250731', 'YYYYMMDD')
  )
group by
  acc_key;
```

PIVOT

Úkol:

Pro každého klienta, který má k 31.12.2024 alespoň jeden z ratingů protistrany ČS, vypište všechny 4, tedy DBRS, FITCH, MOODY, SANDP spolu s názvem klienta. Využijte tabulky PARTIES, PARTY_CLASSF_USAGES, PARTY_CLASSF_TYPES a PARTY_CLASSIFICATIONS.

Řešení:

```
select
  pt_party_name,
  dbrs_rating, fitch_rating, moody_rating, sandp_rating
from
  (
    select
      /*+ leading(t u) use_nl(u p) */
      p.pt_party_name, t.ptclstp_source_id, c.ptcls_source_id
    from
      dwh_owner.party_classf_usages u, dwh_owner.party_classf_types t, dwh_owner.party_classifications c, dwh_owner.parties p
    where
      t.ptclstp_source_id in ('DBRS', 'FITCH', 'MOODY', 'SANDP') and
      u.ptclstp_key = t.ptclstp_key and
      c.ptclstp_key = u.ptclstp_key and
      c.ptcls_key = u.ptcls_key and
      to_date('20241231', 'YYYYMMDD') between u.ptclsus_valid_from and u.ptclsus_valid_to and
      p.ptori_key = 4 and
      u.pt_key = p.pt_key
  )
pivot
  (
    max(ptcls_source_id) as rating for ptclstp_source_id in ('DBRS' as dbrs, 'FITCH' as fitch, 'MOODY' as moody, 'SANDP' as sandp)
  );
```

UNPIVOT

Naopak funkce unpivot slouží k tomu, abychom mohli z více sloupců jednoho řádku udělat více řádků.

Syntaxe:

```
select
*
from
(
  <vnitřní select>
)
unpivot (<nový významový sloupec> for <nový rozlišovací sloupec> in (<názvy sloupců>));
```

Opět si funkci raději ukážeme na příkladu.

UNPIVOT *(07_asql_unpivot_01.txt)*

Příklad:

Z tabulky PARTY_RELATIONS vytvořte dva řádky pro subject a object.

Řešení:

```
select
  ptrel_key,
  pt_key,
  sub_obj
from
  (
    select
      ptrel_key,
      pt_subject_key,
      pt_object_key
    from
      dwh_owner.party_relations
    where
     sysdate between ptrel_valid_from and ptrel_valid_to
  )
unpivot
  (
    pt_key for sub_obj in (pt_subject_key as 'SUBJECT', pt_object_key as 'OBJECT')
  );
```

UNPIVOT

Úkol:

Z jednoho záznamu pro unifikovaného klienta vyrobte tři řádky rozlišující stav klienta - ptst_key, ptstat_cs_key, ptstat_subsidiary_key.

Řešení:

```
select
  pt_key,
  ptst_key,
  ptst_type
from
  (
    select
      pt_key,
      ptst_key,
      ptstat_cs_key,
      ptstat_subsidiary_key
    from
      dwh_owner.parties
    where
     ptori_key = 4
  )
unpivot
  (
    ptst_key for ptst_type in (ptstat_cs_key as 'CS', ptstat_subsidiary_key as 'DCERA', ptst_key as 'FGCS')
  );
```

UNPIVOT *(08_asql_unpivot_02.txt)*

Funkci unpivot můžeme použít i na skupinu atributů, nikoliv na atribut samotný. Například kdybych chtěli u nástrojů s „více nohami“ vytvořit řádky zvlášť pro jednu a druhou nohu.

Příklad:

```
select
  acc_key, acctp_key,
  curr_key, intrtc_key, intrmt_key, noha
from
  (
    select
      acc_key, acctp_key, curr_key, curr_2_key, intrtc_key, intrtc_2_key, intrmt_key, intrmt_2_key
    from
      dwh_owner.treasury_deals
  )
unpivot
  (
    (curr_key, intrtc_key, intrmt_key) for noha in
    (
      (curr_key, intrtc_key, intrmt_key) as '1',
      (curr_2_key, intrtc_2_key, intrmt_2_key) as '2'
    )
  );
```

UNPIVOT

Úkol:

Zjistěte, která nehistorická tabulka obsahuje atributy respct_createdby_key, respct_assignedto_key, respct_primary_key, a pomocí unpivot vypište primární klíč a vyplněné dvojice resp_*_key, respct_*_key.

Řešení:

```
select
  opt_key,
  respct_key,
  resp_key,
  typ
from
  (
    select
      opt_key,
      respct_createdby_key,
      respct_assignedto_key,
      respct_primary_key,
      resp_createdby_key,
      resp_assignedto_key,
      resp_primary_key
    from
      dwh_owner.opportunities
  )
unpivot
(
  (resp_key, respct_key) for typ in
  (
    (resp_createdby_key, respct_createdby_key) as 'createdby',
    (resp_assignedto_key, respct_assignedto_key) as 'assignedto',
    (resp_primary_key, respct_primary_key) as 'primary'
  )
)
where
  resp_key > 0;
```

OPT_KEY	RESPCT_KEY	RESP_KEY	TYP
40877363	17	2671675	assignedto
40877363	17	18763	primary
40877400	17	1833096	assignedto
40877400	17	18763	primary
40877897	17	19125	assignedto
40877897	17	18763	primary
40878335	17	19169	assignedto
40878335	17	19169	primary
40879003	17	17700	assignedto
40879003	17	18763	primary
40879098	17	57993	assignedto
40879098	17	18399	primary

ROLLUP

Funkce ROLLUP rozšiřuje klauzuli GROUP BY a umožňuje nám vypsat i „mezisoučty“, resp. dílčí agregace. Tedy pokud agregujeme podle jediného sloupce, výsledek bude obsahovat i jeden sumární řádek. Například pokud by nás zajímal objem transakcí po měsících roku 2019, tak kromě dvanácti řádků za jednotlivé měsíce bude výsledek obsahovat i třináctý řádek, kde bude celkový objem, tedy objem za celý rok.

Syntaxe:

```
SELECT
  <agregované sloupce>,
  <agregující sloupce>
FROM
  <schéma>.<tabulka>
GROUP BY
   ROLLUP (<agregující sloupce>)
ORDER BY
  <seznam sloupců ASC / DESC>
;
```

Opět si funkci ukažme na příkladu.

ROLLUP *(09_asql_rollup.txt)*

Příklad:

Vypište počet nesmazaných unifikovaných klientů (dívek a chlapců) ve věku 15 – 18 let v závislosti na pohlaví a věku včetně mezisoučtů.

Řešení:

```
select
  pohlavi,
  vek,
  count(1) as pocet
from
  (
    select
      trunc(months_between(sysdate, pt.pt_birth_date) / 12) as vek,
      g.psgen_unified_desc as pohlavi
    from
      dwh_owner.parties pt,
      dwh_owner.person_genders g
    where
      pt.pt_deleted_flag = 'N' and
      pt.ptori_key = 4 and
      pt.psgen_key = g.psgen_key and
      g.psgen_unified_id in ('M', 'Z')
  )
where
  vek between 15 and 18
group by
  rollup(pohlavi, vek)
order by
  pohlavi, vek;
```

POHLAVI	VEK	POCET
MUŽ	15	27458
MUŽ	16	26547
MUŽ	17	26264
MUŽ	18	27335
MUŽ		107604
ŽENA	15	25425
ŽENA	16	24737
ŽENA	17	24556
ŽENA	18	25667
ŽENA		100385
		207989

ROLLUP

Úkol:

Vypište počet kreditních transakcí (ACCFC_MTD_CR_TRANS_CNT v CURRENT_ACCOUNT_FACTS) na všech běžných účtech Symbols za tři předchozí roky po letech, kvartálech a měsících, včetně součtů po ročních kvartálech a po letech.

Řešení:

```
select
  rok,
  ctvrtleti,
  mesic,
  sum(pocet) as pocet
from
  (
    select
      to_char(accfc_period, 'YYYY') as rok,
      to_char(accfc_period, 'Q') as ctvrtleti,
      to_char(accfc_period, 'MM') as mesic,
      accfc_mtd_cr_trans_cnt as pocet
    from
      dwh_owner.current_account_facts
    where
      acctp_key = 602 and
      accfc_period between last_day(add_months(trunc(sysdate, 'YYYY'), -36)) and
      (trunc(sysdate, 'YYYY') - 1)
  )
group by
  rollup(rok, ctvrtleti, mesic)
order by
  rok desc,
  ctvrtleti desc,
  mesic desc;
```

ROK	CTVRTLETI	MESIC	POCET
			235405897
2019			82239722
2019	4		21138583
2019	4	12	7100879
2019	4	11	6986843
2019	4	10	7050861
2019	3		20523323
2019	3	09	6769651
2019	3	08	6685333
2019	3	07	7068339
2019	2		20601704
2019	2	06	6595053
2019	2	05	7071921
2019	2	04	6934730
2019	1		19976112
2019	1	03	6706902
2019	1	02	6385982
2019	1	01	6883228
2018			78876402

CUBE

Funkce CUBE je rozšířením ROLLUP; ta vypisuje všechny mezisoučty, nikoliv jen „zleva doprava“.

Syntaxe:

```
SELECT
  <agregované sloupce>,
  <agregující sloupce>
FROM
  <schéma>.<tabulka>
GROUP BY
   CUBE (<agregující sloupce>)
ORDER BY
  <seznam sloupců ASC / DESC>
;
```

Opět si funkci ukažme na příkladu.

CUBE *(10_asql_cube.txt)*

Příklad:

Vypište počet nesmazaných unifikovaných klientů (dívek a chlapců) ve věku 15 – 18 let v závislosti na pohlaví a věku včetně všech mezisoučtů.

Řešení:

```
select
  pohlavi,
  vek,
  count(1) as pocet
from
  (
    select
      trunc(months_between(sysdate, pt.pt_birth_date) / 12) as vek,
      g.psgen_unified_desc as pohlavi
    from
      dwh_owner.parties pt,
      dwh_owner.person_genders g
    where
      pt.pt_deleted_flag = 'N' and
      pt.ptori_key = 4 and
      pt.psgen_key = g.psgen_key and
      g.psgen_unified_id in ('M', 'Z')
  )
where
  vek between 15 and 18
group by
  cube(pohlavi, vek)
order by
  pohlavi, vek;
```

POHLAVI	VEK	POCET
MUŽ	15	27458
MUŽ	16	26547
MUŽ	17	26264
MUŽ	18	27335
MUŽ		107604
ŽENA	15	25425
ŽENA	16	24737
ŽENA	17	24556
ŽENA	18	25667
ŽENA		100385
	15	52883
	16	51284
	17	50820
	18	53002
		207989

CUBE

Úkol:

Vypište počet kreditních transakcí (ACCFC_MTD_CR_TRANS_CNT v CURRENT_ACCOUNT_FACTS) na všech běžných účtech Symbols za tři předchozí roky po letech, kvartálech a měsících, včetně všech mezisoučtů. Null hodnoty vypisujte jako poslední.

Řešení:

```
select
  rok,
  ctvrtleti,
  mesic,
  sum(pocet) as pocet
from
  (
    select
      to_char(accfc_period, 'YYYY') as rok,
      to_char(accfc_period, 'Q') as ctvrtleti,
      to_char(accfc_period, 'MM') as mesic,
      accfc_mtd_cr_trans_cnt as pocet
    from
      dwh_owner.current_account_facts
    where
      acctp_key = 602 and
      accfc_period between last_day(add_months(trunc(sysdate, 'YYYY'), -36)) and
      (trunc(sysdate, 'YYYY') - 1)
  )
group by
  cube(rok, ctvrtleti, mesic)
order by
  rok desc nulls last,
  ctvrtleti desc nulls last,
  mesic desc nulls last;
```

ROK	CTVRTLETI	MESIC	POCET
2019	4	12	7100879
2019	4	11	6986843
2019	4	10	7050861
2019	4		21138583
2019	3	09	6769651
2019	3	08	6685333
2019	3	07	7068339
2019	3		20523323
2019	2	06	6595053
2019	2	05	7071921
2019	2	04	6934730
2019	2		20601704
2019	1	03	6706902
2019	1	02	6385982
2019	1	01	6883228
2019	1		19976112
2019		12	7100879
2019		11	6986843
2019		10	7050861

GROUPING SETS

Funkce GROUPING SETS je dalším rozšířením ROLLUP; zde si můžeme vybrat, které všechny agregační skupiny chceme vidět. Pokud bychom měli dva agregační sloupce A a B, pak mi existují čtyři možné skupiny. Skupina (), kde agreguji přes celek, skupina (A), kde agreguji přes sloupec A, obdobně (B) přes sloupec B a nakonec (A,B), která agreguje přes oba sloupce. Pomocí GROUPING SETS si můžeme vybrat, kterou skupinu chceme.

ROLLUP odpovídá skupinám (), (A), (A,B), kdežto CUBE obsahuje vždy všechny množiny, tedy (), (A), (B), (A, B).

Syntaxe:

```
SELECT
  <agregované sloupce>,
  <agregující sloupce>
FROM
  <schéma>.<tabulka>
GROUP BY
   GROUPING SETS ((podmnožina_1 z <agregujících sloupců>),..., (podmnožina_2^n z <agregujících sloupců>))
ORDER BY
  <seznam sloupců ASC / DESC>
;
```

Opět si funkci ukažme na příkladu.

GROUPING SETS *(11_asql_grouping_sets.txt)*

Příklad:

Pracujme s množinou nesmazaných unifikovaných klientů (dívek a chlapců) ve věku 15 – 18 let. Vypište jejich celkový počet, počet dle pohlaví, počet dle věku a počet dle prvního písmene příjmení.

Řešení:

```
select
  pohlavi,
  vek,
  prijmeni,
  count(1) as pocet
from
  (
    select
      substr(pt.pt_last_name,1,1) as prijmeni,
      trunc(months_between(sysdate, pt.pt_birth_date) / 12) as vek,
      g.psgen_unified_desc as pohlavi
    from
      dwh_owner.parties pt,
      dwh_owner.person_genders g
    where
      pt.pt_deleted_flag = 'N' and
      pt.ptori_key = 4 and
      pt.psgen_key = g.psgen_key and
      g.psgen_unified_id in ('M', 'Z')
  )
where
  vek between 15 and 18
group by
   grouping sets((), (vek), (pohlavi), (prijmeni))
order by
  pohlavi nulls last, vek nulls last, prijmeni nulls last;
```

POHLAVI	VEK	PRIJMENI	POCET
MUŽ			107603
ŽENA			100385
	15		52883
	16		51284
	17		50819
	18		53002
		A	2027
		Á	6
		B	15283
		C	4259
		Ć	11
		Č	4217
		d	3
		D	8385
		Ď	99
		e	1
		E	828
		É	2
		F	5634

GROUPING SETS

V případě třech agregačních sloupců A, B a C, máme k dispozici celkem 8 různých podmnožin, obecně pro n sloupců máme 2^n možností.

ROLLUP je opět ekvivalentní zápisu GROUPING SETS((), (A), (A,B), (A,B,C)).

CUBE pak GROUPING SETS((), (A), (B), (C), (A,B), (A,C), (B,C),(A,B,C)).

POZOR, v případě GROUPING SETS musí být každý agregující sloupec v alespoň jedné množině!

Poznámka:
Jednosloupcové množiny nemusí být napsány v závorce.

Poznámka:
Množiny se mohou ve výčtu opakovat pro zduplikování řádků.

GROUPING SETS

Úkol:

Vypište počet kreditních transakcí (ACCFC_MTD_CR_TRANS_CNT v CURRENT_ACCOUNT_FACTS) na všech běžných účtech Symbols za tři předchozí roky a to po letech, po kvartálech napříč roky a po měsících napříč roky. Doplňte rovněž celkovou sumu za tři roky.

Řešení:

```
select
  rok,
  ctvrtleti,
  mesic,
  sum(pocet) as pocet
from
  (
    select
      to_char(accfc_period, 'YYYY') as rok,
      to_char(accfc_period, 'Q') as ctvrtleti,
      to_char(accfc_period, 'MM') as mesic,
      accfc_mtd_cr_trans_cnt as pocet
    from
      dwh_owner.current_account_facts
    where
      acctp_key = 602 and
      accfc_period between last_day(add_months(trunc(sysdate, 'YYYY'), -36)) and
      (trunc(sysdate, 'YYYY') - 1)
  )
group by
  grouping sets((), rok, ctvrtleti, mesic)
order by
  rok desc nulls last,
  ctvrtleti desc nulls last,
  mesic desc nulls last;
```

ROK	CTVRTLETI	MESIC	POCET
2019			82239722
2018			78876402
2017			74289773
	4		60984196
	3		57960502
	2		59191270
	1		57269929
		12	20253559
		11	20282601
		10	20448036
		09	18890675
		08	19403629
		07	19666198
		06	19379903
		05	20358320
		04	19453047
		03	19451016
		02	18180338
		01	19638575
			235405897

GROUPING_ID

Pomocí funkce GROUPING_ID můžeme rozlišovat jednotlivé skupiny. V případě, že je parametrem jediný agregující sloupec A, pak funkce vrací 0, pakliže se jedná o konkrétní hodnotu a 1, je-li sumární. V případě sloupců A, B je pak chování následující:

Obě hodnoty existují – vrací 0. Existuje A, B je agregace, pak 1. Existuje B, A je agregace, pak 2. Je to jen sumární záznam, pak 3.

Obecně v případě n sloupců (sl_1, sl_2, …, sl_n) je hodnota spočtena jako suma vah těch sloupců, které nejsou vyplněny (je to dílčí agregace). Váha prvního sloupce je 2^(n-1), n-tého 1 (2^0) a i-tého 2^(n-i).

Syntaxe:

```
SELECT
  <agregované sloupce>,
  <agregující sloupce>,
GROUPING_ID(podmnožina z <agregujících sloupců>)
FROM
  <schéma>.<tabulka>
GROUP BY
  ROLLUP / CUBE / GROUPING SETS (…)
ORDER BY
  <seznam sloupců ASC / DESC>
;
```

GROUPING_ID *(12_asql_grouping_id.txt)*

Příklad:

Pracujme s množinou nesmazaných unifikovaných klientů (dívek a chlapců) ve věku 15 – 18 let. Vypište jejich celkový počet, počet dle pohlaví, počet dle věku a počet dle prvního písmene příjmení.

Řešení:

```
select
  pohlavi,
  vek,
  prijmeni,
  count(1) as pocet,
  grouping_id(pohlavi) as skup_poh,
  grouping_id(vek) as skup_vek,
  grouping_id(prijmeni) as skup_pri,
  grouping_id(pohlavi, vek, prijmeni) as skup_vse
from
  (
    select
      substr(pt.pt_last_name,1,1) as prijmeni,
      trunc(months_between(sysdate, pt.pt_birth_date) / 12) as vek,
      g.psgen_unified_desc as pohlavi
    from
      dwh_owner.parties pt,
      dwh_owner.person_genders g
    where
      pt.pt_deleted_flag = 'N' and
      pt.ptori_key = 4 and
      pt.psgen_key = g.psgen_key and
      g.psgen_unified_id in ('M', 'Z')
  )
where
  vek between 15 and 18
group by
   grouping sets((), (vek), (pohlavi), (prijmeni))
order by
  pohlavi nulls last, vek nulls last, prijmeni nulls last;
```

POHLAVI	VEK	PRIJMENI	POCET	SKUP_POH	SKUP_VEK	SKUP_PRI	SKUP_VSE
MUŽ			107603	0	1	1	3
ŽENA			100385	0	1	1	3
	15		52883	1	0	1	5
	16		51284	1	0	1	5
	17		50819	1	0	1	5
	18		53002	1	0	1	5
		A	2027	1	1	0	6
		Á	6	1	1	0	6
		B	15283	1	1	0	6
		C	4259	1	1	0	6
		Ć	11	1	1	0	6
		Č	4217	1	1	0	6
		d	3	1	1	0	6
		D	8385	1	1	0	6
		Ď	99	1	1	0	6
		e	1	1	1	0	6
		E	828	1	1	0	6
		É	2	1	1	0	6
		F	5634	1	1	0	6
		G	3881	1	1	0	6

Regulární výrazy

Co jsou regulární výrazy?

Regulární výraz, či řetězec, je sekvence znaků s přesně určenou syntaxí, pomocí které můžeme vyhledávat, porovnávat či měnit jiné řetězce.

Jak je můžu využít?

Oracle má 4 základní funkce, které s regulárními řetězci pracují.

REGEXP_LIKE(<řetězec>, <regexp>) – funkce sloužící k porovnání, jestli řetězec odpovídá regulárnímu výrazu. Používá se výhradně v části WHERE.

REGEXP_REPLACE(<zdroj>, <regexp_co>, <za>) – rozšíření funkce REPLACE, ve <zdroji> zamění všechny výskyty odpovídající regulárnímu výrazu <regexp_co> za <za>.

REGEXP_INSTR(<zdroj>, <regexp_co>, <začátek>, <n>) – rozšíření funkce INSTR, vrátí pořadí <n>-tého výskytu řetězce odpovídajícího výrazu <regexp_co> ve <zdroji> počínaje znakem na <začátek>-té pozici.

REGEXP_SUBSTR(<zdroj>, <regexp_co>, <začátek>, <n>) – rozšíření funkce SUBSTR, vrátí pořadí <n>-tého výskytu řetězce odpovídajícího výrazu <regexp_co> ve <zdroji> počínaje znakem na <začátek>-té pozici.

Poznámka:
Funkce mohou být použity s dalším parametrem, kterým se může měnit standardní chování. Typicky se používá ‘i‘ jako „case insensitive“, tedy že nedělám rozdíl mezi malými a velkými písmeny.

Regulární výrazy

Konstrukce regulárních řetězců:

Metaznak	Význam
.	Libovolný znak.
+	Jeden nebo více výskytů znaku / podvýrazu.
*	Nula či více výskytů znaku / podvýrazu.
?	Nula či jeden výskyt znaku / podvýrazu.
{m}	Přesně m výskytů znaku / podvýrazu.
{m,}	Alespoň m znaků / podvýrazů.
{m,n}	Alespoň m a nejvýše n znaků / podvýrazů.
[…]	Místo třech teček je výskyt povolených znaků. Je možné použít i interval pomocí -, např. a-z, A-Z, 0-9.
[^ …]	Místo třech teček je výskyt nepovolených znaků.
|	Oddělovač nebo.
(…)	Označení podvýrazu.
\n	Reference na n-tý podvýraz; používá se zejména u REGEXP_REPLACE.
\	„Escape character“, tedy výraz, který navozuje původní význam znaku. Např. \. značí tečku a nikoliv libovolný znak.
^	Znak pro začátek řetězce.
$	Znak pro konec řetězce.
[:class:]	Třída dle POSIX standardu.
[.element.]	Specifikace „zřetězeného“ znaku.
[=character=]	Specifikace skupiny znaků (odpovídajících diakritice).

Regulární výrazy *(13_asql_regexp_01.txt)*

Příklad:

Vypište všechny nesmazané unifikované klienty (muže a ženy), kteří mají více než jedno křestní jméno či příjmení.

Řešení:

```
select
  pt_party_name
from
  dwh_owner.parties
where
  psgen_key in (1, 2) and
  ptori_key = 4 and
  pt_deleted_flag = 'N' and
  regexp_like(pt_party_name, '^[^ ]* .+ [^ ]*$');
```

PT_PARTY_NAME
Van Der Lindenová Hana
Bulová Konečná Gabriela
Kaděrová Ing. Ivana
Abdul Hamid Ahmed
Paniagua Carrillo Henry
Dražilová Svačinová Lenka
Horaková Filipová Marta
Reinsch Staffová Milena
Khuc Huu Hung
Ing. Jan Blažík
Nováčková Tilkovská Petra
Oroszová Pátková Jana
Ťápal Ing. Jaroslav
Zatloukalová Jana Florentýna
Roubalová Kostlánová Věra
Bodin Rytířová Lenka
Ševčíková Iva Anežka
Hla Shwe Maung
Baudyšová Jirků Maria
Šilhánek František Václav

Regulární výrazy *(14_asql_regexp_02.txt)*

Příklad:

U klientů z minulého příkladu i toto „další“ jméno vypište.

Řešení:

```
select
  pt_party_name,
  regexp_replace(pt_party_name, '(^[^ ]*) (.+) ([^ ]*$)', '\2') repl,
  regexp_substr(pt_party_name, ' .+ ') subs
from
  dwh_owner.parties
where
  psgen_key in (1, 2) and
  ptori_key = 4 and
  pt_deleted_flag = 'N' and
  regexp_like(pt_party_name, '^[^ ]* .+ [^ ]*$');
```

PT_PARTY_NAME	REPL	SUBS
Tran Hai Ninh	Hai	Hai
Janečková tomášová Ilona	tomášová	tomášová
Obršlíková Procházková Eva	Procházková	Procházková
Tarkó Helena Marie	Helena	Helena
Larišová Markéta Olga	Markéta	Markéta
Nour Mohammadi Beata	Mohammadi	Mohammadi
Junek Horymír Kryštof	Horymír	Horymír
Jandová Minxová Ludmila	Minxová	Minxová
Laiblová Kadlecová Ivana	Kadlecová	Kadlecová
Herianová Bernatová Renáta	Bernatová	Bernatová
Groll Petridesová Věra	Petridesová	Petridesová
Belling Muškátová Iva	Muškátová	Muškátová
Lhotová Marie Gabriela	Marie	Marie
Hartmannová Kybalová Ludmila	Kybalová	Kybalová
Enachescu Hroncová Iva	Hroncová	Hroncová
Charvátová Jaroslava Vlasta	Jaroslava	Jaroslava
Valdéz Bruno Filip	Bruno	Bruno
Drnková Nemejovská Ivona	Nemejovská	Nemejovská
Pešek-Janowsky Oldřich Josef	Oldřich	Oldřich
Langová Šindelářová Marcela	Šindelářová	Šindelářová

Regulární výrazy *(15_asql_regexp_03.txt)*

Příklad:

U klientů z minulého příkladu i toto „další“ jméno vypište; místo první mezery pracujte i s pomlčkou jako oddělovačem.

Řešení:

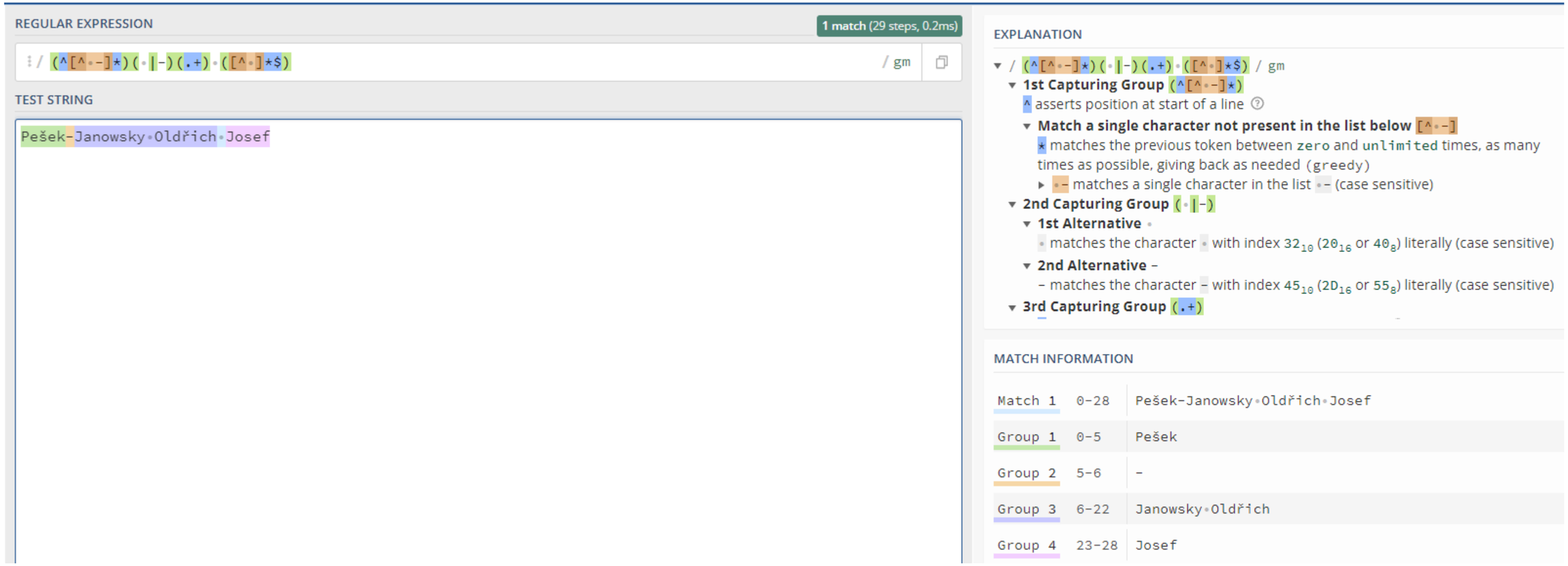
```
select
  pt_party_name,
  regexp_replace(pt_party_name, '(^[^ -]*)( |-)(.+) ([^ ]*$)', '\3') repl,
  regexp_substr(pt_party_name, '( |-).+ ') subs
from
  dwh_owner.parties
where
  psgen_key in (1, 2) and
  ptori_key = 4 and
  pt_deleted_flag = 'N' and
  regexp_like(pt_party_name, '^[^ -]*( |-).+ [^ ]*$');
```

PT_PARTY_NAME	REPL	SUBS
Tran Hai Ninh	Hai	Hai
Al-Hajjarová Anna	Hajjarová	-Hajjarová
Janečková tomášová Ilona	tomášová	tomášová
Obršlíková Procházková Eva	Procházková	Procházková
Tarkó Helena Marie	Helena	Helena
Larišová Markéta Olga	Markéta	Markéta
Nour Mohammadi Beata	Mohammadi	Mohammadi
Junek Horymír Kryštof	Horymír	Horymír
Jandová Minxová Ludmila	Minxová	Minxová
Laiblová Kadlecová Ivana	Kadlecová	Kadlecová
Herianová Bernatová Renáta	Bernatová	Bernatová
Groll Petridesová Věra	Petridesová	Petridesová
Belling Muškátová Iva	Muškátová	Muškátová
Lhotová Marie Gabriela	Marie	Marie
Hartmannová Kybalová Ludmila	Kybalová	Kybalová
Enachescu Hroncová Iva	Hroncová	Hroncová
Charvátová Jaroslava Vlasta	Jaroslava	Jaroslava
Valdéz Bruno Filip	Bruno	Bruno
Drnková Nemejovská Ivona	Nemejovská	Nemejovská
Pešek-Janowsky Oldřich Josef	Janowsky Oldřich	-Janowsky Oldřich

Regulární výrazy

Poznámka:

Existují i online „ověřovače“ regulárních výrazů, které vám ověří syntaxi a zároveň udělají rozpad na jednotlivé podvýrazy. Jeden z nich najdete na adrese https://regex101.com/.



Regulární výrazy

Úkol:

Vypište všechny nesmazané unifikované klienty (muže a ženy) spolu s jejich mailem (typ W-E-MAIL či E-MAIL), který bez ohledu na velká a malá písmena končí csas.cz, ext.csas.cz nebo csps.cz. Použijte regulární výraz. Pro test pracujte jen s klíči (contp_key = 345645541 or contp_key between 352667702 and 359327484).

Řešení:

```
select
    pt.pt_party_name as jmeno,
    cpt.contptp_unified_id as typ,
    cp.contp_value as mail
  from
    dwh_owner.parties pt,
    dwh_owner.contact_points cp,
    dwh_owner.contact_point_types cpt
  where
    pt.psgen_key in (1, 2) and
    pt.ptori_key = 4 and
    pt.pt_deleted_flag = 'N' and
    pt.pt_key = cp.pt_key and
    cp.ptori_key = 4 and
    cp.contptp_key = cpt.contptp_key and
    cpt.contptp_unified_id in ('E-MAIL', 'W-E-MAIL') and
    sysdate between cp.contp_valid_from and cp.contp_valid_to and
    regexp_like(cp.contp_value, '^.+@(ext\.)?(csas|csps)\.cz$', 'i');
```

JMENO	TYP	MAIL
Klímová Jana	W-E-MAIL	jaklimova@csas.cz
Preininger Miloslav	E-MAIL	cen8112dedickarizeniumorovanivk@csas.cz
Tříska Dušan	E-MAIL	cen8432am@csas.cz
Krýslová Eva	W-E-MAIL	ekryslova@csas.cz
Nykl Tomáš	E-MAIL	tnykl@csas.cz
Moucha Zdeněk	E-MAIL	skralickova@csas.cz
Pešek Miroslav	E-MAIL	jitpeskova@csas.cz
Straka Jiří	E-MAIL	rpecinkova@csas.cz
Pflegerová Dagmar	E-MAIL	rtrestikova@csas.cz
Sladká Dagmar	E-MAIL	dagmar.sladka@ext.csas.cz
Jüttner František	E-MAIL	fjuttner@csas.cz
Zámyslický Petr	E-MAIL	pzamyslicky@csas.cz
Benetka Miloš	E-MAIL	lpasek@csas.cz
Skorkovská Helena	E-MAIL	hskorkovska@csas.cz
Keslová Jana	E-MAIL	pkubiskova@csas.cz
Skokánková Eva	E-MAIL	eskokankova@csas.cz
Liška Petr	E-MAIL	peliska@csas.cz

Regulární výrazy

Úkol:

Z minulého případu vypište pouze ty maily, jež mají tvar „1. písmeno křestního jména a poté příjmení bez diakritiky“. Té se zbavíte např. použitím convert(jmeno, ‘us7ascii‘).

Řešení:

```
select
    pt.pt_party_name as jmeno,
    cpt.contptp_unified_id as typ,
    cp.contp_value as mail
  from
    dwh_owner.parties pt,
    dwh_owner.contact_points cp,
    dwh_owner.contact_point_types cpt
  where
    pt.psgen_key in (1, 2) and
    pt.ptori_key = 4 and
    pt.pt_deleted_flag = 'N' and
    pt.pt_key = cp.pt_key and
    cp.ptori_key = 4 and
    cp.contptp_key = cpt.contptp_key and
    cpt.contptp_unified_id in ('E-MAIL', 'W-E-MAIL') and
    sysdate between cp.contp_valid_from and cp.contp_valid_to and
    regexp_like(cp.contp_value, '^'||convert(substr(pt.pt_first_name,1,1), 'us7ascii') ||convert(pt.pt_last_name, 'us7ascii')||
'@(ext\.)?(csas|csps)\.cz$', 'i');
```

JMENO	TYP	MAIL
Krýslová Eva	W-E-MAIL	ekryslova@csas.cz
Nykl Tomáš	E-MAIL	tnykl@csas.cz
Jüttner František	E-MAIL	fjuttner@csas.cz
Zámyslický Petr	E-MAIL	pzamyslicky@csas.cz
Skorkovská Helena	E-MAIL	hskorkovska@csas.cz
Skokánková Eva	E-MAIL	eskokankova@csas.cz
Stejskalová Anna	E-MAIL	astejskalova@csas.cz
Řehák Jiří	E-MAIL	jrehak@csas.cz
Charypar Petr	E-MAIL	PCHARYPAR@CSAS.CZ
Mečířová Hana	E-MAIL	hmecirova@csas.cz
Kohoušek Ivo	E-MAIL	ikohousek@csas.cz
Koucká Stanislava	W-E-MAIL	skoucka@csas.cz
Rameš Antonín	W-E-MAIL	arames@csas.cz
Pytlounová Lenka	E-MAIL	lpytlounova@csas.cz
Procházka Jan	W-E-MAIL	jprochazka@csas.cz
Kubát František	W-E-MAIL	fkubat@csas.cz
Pospíchalová Hana	E-MAIL	hpospichalova@csas.cz

Regulární výrazy

Úkol:

Výsledek z minula přepište pomocí funkce PIVOT tak, že E-MAIL bude sloupec označený soukromy_mail a W-E-MAIL jako pracovni_mail.

Řešení:

```
select
  jmeno, soukromy_mail, pracovni_mail
from
  (
    select
      pt.pt_party_name as jmeno,
      cpt.contptp_unified_id as typ,
      cp.contp_value as mail
    from
      dwh_owner.parties pt,
      dwh_owner.contact_points cp,
      dwh_owner.contact_point_types cpt
    where
      pt.psgen_key in (1, 2) and
      pt.ptori_key = 4 and
      pt.pt_deleted_flag = 'N' and
      pt.pt_key = cp.pt_key and
      cp.ptori_key = 4 and
      cp.contptp_key = cpt.contptp_key and
      cpt.contptp_unified_id in ('E-MAIL', 'W-E-MAIL') and
      sysdate between cp.contp_valid_from and cp.contp_valid_to and
      regexp_like(cp.contp_value, '^'||convert(substr(pt.pt_first_name,1,1), 'us7ascii') ||convert(pt.pt_last_name, 'us7ascii')|| '@(ext\.)?(csas|csps)\.cz$', 'i')
  )
pivot
  (max (mail) as mail for typ in ('E-MAIL' as soukromy, 'W-E-MAIL' as pracovni));
```

JMENO	SOUKROMY_MAIL	PRACOVNI_MAIL
Bánovská Jana	jbanovska@csas.cz	
Bouška Zdeněk	zbouska@csas.cz	
Strýček Aleš	astrycek@csas.cz	
Mlynárová Radka	rmlynarova@csas.cz	
Franče Václav	vfrance@csas.cz	
Krajníková Marta	MKRAJNIKOVA@CSAS.CZ	
Derner Petr	pderner@csas.cz	
Kusáková Eva	ekusakova@csas.cz	
Kleibl Milan	MKLEIBL@CSAS.CZ	
Vlažná Ivona	ivlazna@csas.cz	
Michalec Petr		pmichalec@csas.cz
Randýsková Kristýna	krandyskova@csas.cz	

Regulární výrazy

POSIX třídy:

Metaznak	Význam
[:alnum:]	Všechny alfanumerické znaky.
[:alpha:]	Všechny alfabetické znaky.
[:blank:]	Všechny „bílé“ tisknutelné znaky – tj. mezera a tabulátor.
[:cntrl:]	Všechny kontrolní, tedy netisknutelné znaky.
[:digit:]	Všechny numerické znaky.
[:graph:]	Všechny znaky z tříd [:punct:], [:upper:], [:lower:] a [:digit:].
[:lower:]	Všechna malá písmena abecedy.
[:print:]	Všechny tisknutelné znaky
[:punct:]	Všechny interpunkční znaky.
[:space:]	Všechny „bílé“ netisknutelné znaky – např. konec řádky atd.
[:upper:]	Všechna velká písmena abecedy.
[:xdigit:]	Všechny hexadecimální znaky.

Regulární výrazy

[.element.]

Díky této konstrukci můžeme definovat zřetězený znak, např. v češtině ch. Poté se dvojznak chová jako jeden a funguje i např. v intervalu. [a-[.ch.]] definuje znaky a, b, c, d, e, f, g, h, ch (včetně diakritiky).

Pro správnou funkčnost je třeba spustit ještě

alter session set nls_sort = xczech;

[=character=]

Definuje „podobné“ znaky, např. [=a=] zastřešuje znaky a, á, ä, â.

Regulární výrazy *(16_asql_regexp_04.txt)*

Příklad:

Ze všech platných soukromých a služebních telefonů z CRM, které obsahují jiné znaky než číslice, tyto znaky vyhoďte.

Řešení:

```
select
  contp_value as originalni,
  regexp_replace(contp_value, '[^[:digit:]]', '') as upraveny
from
  dwh_owner.contact_points
where
  ptori_key = 4 and
  contptp_key in (6, 13) and
  sysdate between contp_valid_from and contp_valid_to and
  not regexp_like(contp_value, '^[[:digit:]]*$');
```

ORIGINALNI	UPRAVENY
+919908118905	919908118905
+421948364739	421948364739
+4210908033755	4210908033755
+421940776315	421940776315
+421903729281	421903729281
+421918286080	421918286080
+16238769376	16238769376
+421910137173	421910137173
+421948106678	421948106678
+2304648069	2304648069
+421915284915	421915284915
+421944475709	421944475709
+37128480000	37128480000

Regulární výrazy

Úkol:

Pracujme opět s telefonními čísly z minulého příkladu. Vyberte ty, které začínají 00420, +420, 420 a poté následuje 9 číslic nebo obsahují jen 9 číslic. Předpokládejte, že po předčíslí i každé trojici může, ale nemusí být mezera. Číslo vypište ve formátu +420 xxx xxx xxx.

Řešení:

```
select
  contp_value as originalni,
  regexp_replace(contp_value, '^((00|\+)?420)?[[:blank:]]?([[:digit:]]{3})[[:blank:]]?([[:digit:]]{3})[[:blank:]]?([[:digit:]]{3})$', '+420 \3 \4 \5') as uprav
from
  dwh_owner.contact_points
where
  ptori_key = 4 and
  contptp_key in (6, 13) and
  sysdate between contp_valid_from and contp_valid_to and
  regexp_like(contp_value, '^((00|\+)?420)?([[:blank:]]?[[:digit:]]{3}){3}$');
```

ORIGINALNI	UPRAVENY
731633687	+420 731 633 687
737412479	+420 737 412 479
728514674	+420 728 514 674
352603013	+420 352 603 013
602297951	+420 602 297 951
777181777	+420 777 181 777
602211114	+420 602 211 114
723414256	+420 723 414 256
724809284	+420 724 809 284

Hierarchické dotazy

Jedná se o mechanismus, díky kterému můžu tabulku / poddotaz navazovat sám na sebe a vytvořit tak hierarchie. Typickým příkladem bývá vztah zaměstnanec – manažer, kde manažer je rovněž zaměstnancem, který má svého nadřízeného.

Syntaxe:

```
SELECT
  *
FROM
  <schéma>.<tabulka>
(START WITH <podmínka>)
CONNECT BY (NOCYCLE)  PRIOR <„parent“ sloupec> = <„child“ sloupec>
(ORDER SIBLINGS BY <sloupec>)
;
```

Máme k dispozici i nové funkce / „pseudo“ sloupce.

LEVEL – úroveň zanoření, kořen má 1
CONNECT_BY_ISCYCLE – vrací 1, jestliže tento řádek způsobuje cyklus; jinak 0
CONNECT_BY_ISLEAF - vrací 1, jestliže se jedná o list stromu; jinak 0
SYS_CONNECT_BY_PATH(<sloupec>, <oddělovač>) – vrací hodnoty sloupců všech záznamů v hierarchii oddělené oddělovačem
CONNECT_BY_ROOT – vrací hodnotu sloupce z kořenového záznamu

Hierarchické dotazy *(17_asql_hier_01.txt)*

Příklad:

Klauzule CONNECT BY se nemusí aplikovat jen na sloupce, je možné použít i podmínku na LEVEL. Takto například vypíšeme čísla od 1 do 10.

Řešení:

```
select
  level as cislo
from
  dual
connect by
  level <= 10;
```

CISLO
1
2
3
4
5
6
7
8
9
10

Hierarchické dotazy

Úkol:

Zkuste obdobným způsobem jako v minulém případě vypsat všechny dny aktuálního roku.

Řešení:

```
select
  trunc(sysdate, 'YYYY') + level - 1 as den
from
  dual
connect by
  level <= 365 +
  (case
    when
      mod(to_number(to_char(sysdate, 'YYYY')), 400) = 0
    then
      1
    when
      mod(to_number(to_char(sysdate, 'YYYY')), 100) = 0
    then
      0
    when
      mod(to_number(to_char(sysdate, 'YYYY')), 4) = 0
    then
      1
    else
      0
    end
  );
```

DEN
01.01.2020
02.01.2020
03.01.2020
04.01.2020
05.01.2020
06.01.2020
07.01.2020
08.01.2020
09.01.2020
10.01.2020
11.01.2020
12.01.2020
13.01.2020

Hierarchické dotazy *(18_asql_hier_02.txt)*

Příklad:

Vypište všechny zaměstnance, kteří v organizační struktuře spadají pod Danielu Peškovou.

Řešení:

```
select
  pt_party_name,
  connect_by_root(pt_party_name),
  sys_connect_by_path(pt_party_name, '/')
from
  (
    select
      e.emp_key,
      e.emp_manager_key,
      p.pt_party_name
    from
       dwh_owner.employees e,
       dwh_owner.parties p
     where
       e.emp_deleted_flag = 'N' and
       sysdate between e.emp_hire_date and e.emp_leave_date and
       p.ptori_key = 11 and
       e.pt_key = p.pt_key
  )
 start with emp_key = 26209
 connect by prior emp_key = emp_manager_key
 order by pt_party_name;
```

PT_PARTY_NAME	CONNECT_BY_ROOT(PT_PARTY_NAME)	SYS_CONNECT_BY_PATH(PT_PARTY_NAME,'/')
Abbrentová Jindřiška	Pešková Daniela	/Pešková Daniela/Blaha Marek/Peškař Petr/Ropický Petr/Krejčová Jitka/Kubištová Ivona/Abbrentová Jindřiška
Aberle Martin	Pešková Daniela	/Pešková Daniela/Blaha Marek/Peškař Petr/Kříženecká Hana/Němec Marek/Veselský Daniel/Aberle Martin
Aberle Milan	Pešková Daniela	/Pešková Daniela/Šamonilová Monika/Aberle Milan
Aberlová Eva	Pešková Daniela	/Pešková Daniela/Blaha Marek/Peškař Petr/Kříženecká Hana/Honzíček Jiří/Kouřilová Pavla/Aberlová Eva
Abrahamová Pavla	Pešková Daniela	/Pešková Daniela/Blaha Marek/Peškař Petr/Šuhájek Jiří/Obšil Radoslav/Abrahamová Pavla
Abrhámová Jolana	Pešková Daniela	/Pešková Daniela/Blaha Marek/Petr Tomáš/Forst Jiří/Abrhámová Jolana
Absolonová Miroslava	Pešková Daniela	/Pešková Daniela/Blaha Marek/Peškař Petr/Karpetová Iveta/Matušek Roman/Absolonová Miroslava
Adam Jana	Pešková Daniela	/Pešková Daniela/Blaha Marek/Peškař Petr/Michalovičová Michaela/Šauer Jiří/Petroušová Hana/Adam Jana
Adamczyková Denisa	Pešková Daniela	/Pešková Daniela/Blaha Marek/Peškař Petr/Karpetová Iveta/Mikulcová Iveta/Bojarská Monika/Adamczyková Denisa

Hierarchické dotazy *(19_asql_hier_03.txt)*

Příklad:

Podívejte se, jak výpis změní klauzule „SIBLINGS“.

Řešení:

```
select
  pt_party_name,
  connect_by_root(pt_party_name),
  sys_connect_by_path(pt_party_name, '/')
from
  (
    select
      e.emp_key,
      e.emp_manager_key,
      p.pt_party_name
    from
       dwh_owner.employees e,
       dwh_owner.parties p
     where
       e.emp_deleted_flag = 'N' and
       sysdate between e.emp_hire_date and e.emp_leave_date and
       p.ptori_key = 11 and
       e.pt_key = p.pt_key
  )
 start with emp_key = 26209
 connect by prior emp_key = emp_manager_key
 order siblings by pt_party_name;
```

PT_PARTY_NAME	CONNECT_BY_ROOT(PT_PARTY_NAME)	SYS_CONNECT_BY_PATH(PT_PARTY_NAME,'/')
Pešková Daniela	Pešková Daniela	/Pešková Daniela
Belant Filip	Pešková Daniela	/Pešková Daniela/Belant Filip
Bartoňová Hana	Pešková Daniela	/Pešková Daniela/Belant Filip/Bartoňová Hana
Bartošová Renáta	Pešková Daniela	/Pešková Daniela/Belant Filip/Bartošová Renáta
Bašovská Sandra	Pešková Daniela	/Pešková Daniela/Belant Filip/Bašovská Sandra
Boháčová Jitka	Pešková Daniela	/Pešková Daniela/Belant Filip/Boháčová Jitka
Budiš Miroslav	Pešková Daniela	/Pešková Daniela/Belant Filip/Budiš Miroslav
Coufalová Michaela	Pešková Daniela	/Pešková Daniela/Belant Filip/Coufalová Michaela

Hierarchické dotazy *(20_asql_hier_04.txt)*

Příklad:

Podívejte se, jak funguje CONNECT_BY_ISLEAF.

Řešení:

```
select
  pt_party_name,
  connect_by_isleaf,
  sys_connect_by_path(pt_party_name, '/')
from
  (
    select
      e.emp_key,
      e.emp_manager_key,
      p.pt_party_name
    from
       dwh_owner.employees e,
       dwh_owner.parties p
     where
       e.emp_deleted_flag = 'N' and
       sysdate between e.emp_hire_date and e.emp_leave_date and
       p.ptori_key = 11 and
       e.pt_key = p.pt_key
  )
 start with emp_key = 26209
 connect by prior emp_key = emp_manager_key
 order siblings by pt_party_name;
```

PT_PARTY_NAME	CONNECT_BY_ISLEAF	SYS_CONNECT_BY_PATH(PT_PARTY_NAME,'/')
Pešková Daniela	0	/Pešková Daniela
Belant Filip	0	/Pešková Daniela/Belant Filip
Bartoňová Hana	1	/Pešková Daniela/Belant Filip/Bartoňová Hana
Bartošová Renáta	1	/Pešková Daniela/Belant Filip/Bartošová Renáta
Bašovská Sandra	1	/Pešková Daniela/Belant Filip/Bašovská Sandra
Boháčová Jitka	1	/Pešková Daniela/Belant Filip/Boháčová Jitka
Budiš Miroslav	1	/Pešková Daniela/Belant Filip/Budiš Miroslav
Coufalová Michaela	1	/Pešková Daniela/Belant Filip/Coufalová Michaela
Cvrček Jaroslav	1	/Pešková Daniela/Belant Filip/Cvrček Jaroslav

Hierarchické dotazy *(21_asql_hier_05.txt)*

Příklad:

A na jaké úrovni (levelu) v hierarchii je.

Řešení:

```
select
  pt_party_name,
  level,
  sys_connect_by_path(pt_party_name, '/')
from
  (
    select
      e.emp_key,
      e.emp_manager_key,
      p.pt_party_name
    from
       dwh_owner.employees e,
       dwh_owner.parties p
     where
       e.emp_deleted_flag = 'N' and
       sysdate between e.emp_hire_date and e.emp_leave_date and
       p.ptori_key = 11 and
       e.pt_key = p.pt_key
  )
 start with emp_key = 26209
 connect by prior emp_key = emp_manager_key
 order by pt_party_name;
```

PT_PARTY_NAME	LEVEL	SYS_CONNECT_BY_PATH(PT_PARTY_NAME,'/')
Abbrentová Jindřiška	7	/Pešková Daniela/Blaha Marek/Peškař Petr/Ropický Petr/Krejčová Jitka/Kubištová Ivona/Abbrentová Jindřiška
Aberle Martin	7	/Pešková Daniela/Blaha Marek/Peškař Petr/Kříženecká Hana/Němec Marek/Veselský Daniel/Aberle Martin
Aberle Milan	3	/Pešková Daniela/Šamonilová Monika/Aberle Milan
Aberlová Eva	7	/Pešková Daniela/Blaha Marek/Peškař Petr/Kříženecká Hana/Honzíček Jiří/Kouřilová Pavla/Aberlová Eva
Abrahamová Pavla	6	/Pešková Daniela/Blaha Marek/Peškař Petr/Šuhájek Jiří/Obšil Radoslav/Abrahamová Pavla
Abrhámová Jolana	5	/Pešková Daniela/Blaha Marek/Petr Tomáš/Forst Jiří/Abrhámová Jolana
Absolonová Miroslava	6	/Pešková Daniela/Blaha Marek/Peškař Petr/Karpetová Iveta/Matušek Roman/Absolonová Miroslava
Adam Jana	7	/Pešková Daniela/Blaha Marek/Peškař Petr/Michalovičová Michaela/Šauer Jiří/Petroušová Hana/Adam Jana
Adamczyková Denisa	7	/Pešková Daniela/Blaha Marek/Peškař Petr/Karpetová Iveta/Mikulcová Iveta/Bojarská Monika/Adamczyková Denisa
Adamczyková Veronika	6	/Pešková Daniela/Blaha Marek/Peškař Petr/Pokorná Dagmar/Kolmajer Tomáš/Adamczyková Veronika

Hierarchické dotazy

Úkol:

Sestavte kontrolingovou organizační strukturu (org_source_system_id = 'CG0' and org_master_ctrl_key) počínaje org_key = 564988 a vypište ty jednotky, které mají nějakou podřízenou a zároveň se vyskytují nejdříve pět úrovní pod počáteční pobočkou. Pozor, počáteční pobočka má vazbu sama na sebe!

Řešení:

```
select
  org_name,
  sys_connect_by_path(org_name, '->') as cesta
from
  dwh_owner.organizations
where
  org_source_system_id = 'CG0' and
  connect_by_isleaf = 0 and
  level >= 6
start with
  org_key = 564988
connect by nocycle
  prior org_key = org_master_ctrl_key;
```

ORG_NAME	CESTA
GROUP REAL ESTATE	->BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->REAL ESTATE->GROUP REAL ESTATE
TOP SME	->BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->TOP SME
RKC	->BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->RKC
4222 RKC PLZEŇ	->BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->RKC->4222 RKC PLZEŇ
4223 RKC ÚSTÍ NAD LABEM	->BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->RKC->4223 RKC ÚSTÍ NAD LABEM
4224 RKC LIBEREC	->BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->RKC->4224 RKC LIBEREC
4225 RKC HRADEC KRÁLOVÉ	->BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->RKC->4225 RKC HRADEC KRÁLOVÉ
4226 RKC PARDUBICE	->BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->RKC->4226 RKC PARDUBICE
4227 RKC ČESKÉ BUDĚJOVICE	->BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->RKC->4227 RKC ČESKÉ BUDĚJOVICE
4228 RKC JIHLAVA	->BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->RKC->4228 RKC JIHLAVA

Hierarchické dotazy

Úkol:

Z tabulky dwh_owner.gpm_prod_hierarchy_relations sestavte hierarchii (počínaje záznamy bez nadřízeného) typu „PRODUKTOVÝ KATALOG GPM PRO LDS“. Vypište posloupnost gpmphnd_source_id.

Řešení:

```
select
  gpmphnd_source_id,
  level,
  sys_connect_by_path(gpmphnd_source_id, '/') as cesta
from
  (
    select
      n.gpmphnd_source_id,
      r.gpmphnd_key,
      r.gpmphnd_master_key
    from
      dwh_owner.gpm_prod_hierarchy_relations r,
      dwh_owner.gpm_prod_hierarchy_nodes n
    where
      sysdate between r.gpmphrl_valid_from and
      r.gpmphrl_valid_to and
      r.gpmphtp_key = 42 and
      r.gpmphnd_key = n.gpmphnd_key
  )
connect by
  prior gpmphnd_key = gpmphnd_master_key
start with
  gpmphnd_master_key = -1;
```

GPMPHND_SOURCE_ID	LEVEL	CESTA
AL10000	1	/AL10000
BL10100	2	/AL10000/BL10100
CL10103	3	/AL10000/BL10100/CL10103
103000LA	4	/AL10000/BL10100/CL10103/103000LA
160458	5	/AL10000/BL10100/CL10103/103000LA/160458
103000OA	4	/AL10000/BL10100/CL10103/103000OA
110100	5	/AL10000/BL10100/CL10103/103000OA/110100
110101	5	/AL10000/BL10100/CL10103/103000OA/110101
110102	5	/AL10000/BL10100/CL10103/103000OA/110102
110108	5	/AL10000/BL10100/CL10103/103000OA/110108
110109	5	/AL10000/BL10100/CL10103/103000OA/110109
110110	5	/AL10000/BL10100/CL10103/103000OA/110110
110111	5	/AL10000/BL10100/CL10103/103000OA/110111
120103	5	/AL10000/BL10100/CL10103/103000OA/120103
120104	5	/AL10000/BL10100/CL10103/103000OA/120104
120105	5	/AL10000/BL10100/CL10103/103000OA/120105
120108	5	/AL10000/BL10100/CL10103/103000OA/120108
120109	5	/AL10000/BL10100/CL10103/103000OA/120109

Hierarchické dotazy – rekurzivní WITH

Podobnou funkcionalitu nám zajišťuje i rekurzivní with, který má složitější syntaxi, ale bývá pochopitelnější a rychlejší.

Syntaxe:

```
WITH
 <tabulka_h> (<sloupce>) as
(
 SELECT
  <sloupce>
 FROM
  <tabulka>
 WHERE
  <podmínka pro start>
 UNION ALL
 SELECT
  <sloupce>
 FROM
  <tabulka> t, <tabulka_h> h
 WHERE
  t.<fk> = h.<pk>
)
SELECT
 *
FROM
 <tabulka_h>;
```

Hierarchické dotazy – rekur. WITH *(22_asql_rec_with_01.txt)*

Příklad:

Vypište všechny zaměstnance, kteří v organizační struktuře spadají pod Danielu Peškovou.

Řešení:

```
with
  zamestnanci
as
  (
    select
      pt_party_name as jmeno,
      emp_key, emp_manager_key
    from
      dwh_owner.employees e, dwh_owner.parties p
     where
       e.emp_deleted_flag = 'N' and
       sysdate between e.emp_hire_date and e.emp_leave_date and
       p.ptori_key = 11 and e.pt_key = p.pt_key
  ),
  zamestnanci_h (jmeno, cesta, emp_key, emp_manager_key, lvl)
as
  (
    select
      jmeno, jmeno as cesta,
      emp_key,  emp_manager_key, 1 as lvl
    from
      zamestnanci
    where
      emp_key = 26209
    union all
    select
      z.jmeno, pred.cesta || '->' || z.jmeno as cesta,
      z.emp_key,  z.emp_manager_key, pred.lvl + 1 as lvl
    from
      zamestnanci z, zamestnanci_h pred
    where
      z.emp_manager_key = pred.emp_key
  )
select
  *
from
  zamestnanci_h
order by
  cesta
;
```

JMENO	CESTA	EMP_KEY	EMP_MANAGER_KEY	LVL
Pešková Daniela	Pešková Daniela	26209	86149	1
Belant Filip	Pešková Daniela->Belant Filip	16704	26209	2
Bartoňová Hana	Pešková Daniela->Belant Filip->Bartoňová Hana	218436	16704	3
Bartošová Renáta	Pešková Daniela->Belant Filip->Bartošová Renáta	6262	16704	3
Bašovská Sandra	Pešková Daniela->Belant Filip->Bašovská Sandra	250230	16704	3
Boháčová Jitka	Pešková Daniela->Belant Filip->Boháčová Jitka	204828	16704	3
Budiš Miroslav	Pešková Daniela->Belant Filip->Budiš Miroslav	255034	16704	3
Coufalová Michaela	Pešková Daniela->Belant Filip->Coufalová Michaela	256212	16704	3
Cvrček Jaroslav	Pešková Daniela->Belant Filip->Cvrček Jaroslav	205519	16704	3
Červená Aneta	Pešková Daniela->Belant Filip->Červená Aneta	204709	16704	3

Hierarchické dotazy

Úkol:

Sestavte kontrolingovou organizační strukturu (org_source_system_id = 'CG0' and org_master_ctrl_key) počínaje org_key = 564988 a vypište ty jednotky, které se vyskytují nejdříve pět úrovní pod počáteční pobočkou. Pozor, počáteční pobočka má vazbu sama na sebe! Použijte rekurzivní WITH.

Řešení:

```
with
  organizations_h (org_key, org_master_ctrl_key, org_name, cesta, org_source_system_id, lvl)
as
  (
    select
      org_key,
      org_master_ctrl_key,
      org_name,
      org_name as cesta,
      org_source_system_id,
      1 as lvl
    from
      dwh_owner.organizations
    where
      org_key = 564988
    union all
    select
      o.org_key,
      o.org_master_ctrl_key,
      o.org_name,
      h.cesta || '->' || o.org_name as cesta,
      o.org_source_system_id,
      h.lvl + 1 as lvl
    from
      dwh_owner.organizations o,
      organizations_h h
    where
      o.org_master_ctrl_key = h.org_key and
      o.org_master_ctrl_key <> o.org_key
  )
select
  org_name,
  cesta
from
  organizations_h
where
  org_source_system_id = 'CG0' and
  lvl >= 6;
```

ORG_NAME	CESTA
GROUP REAL ESTATE	BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->REAL ESTATE->GROUP REAL ESTATE
4210 SPECIÁLNÍ PROGRAMY	BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->4210 SPECIÁLNÍ PROGRAMY
SFU-AGRO	BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->SFU-AGRO
TOP SME	BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->TOP SME
RKC	BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->RKC
OTHER CORPORATE	BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->CORPORATE CLIENTS->OTHER CORPORATE
GLC HOLDING	BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->LARGE CORPORATE->GLC HOLDING
ALCO PORTFOLIO - DLUHOPISY GLC	BEZ POBOČKY->CS GROUP->CSAS->CORPORATE->LARGE CORPORATE->ALCO PORTFOLIO - DLUHOPISY GLC

Analytické funkce

Analytické funkce na rozdíl od funkcí agregačních nijak nemění počet vrácených řádků, přesto mohou nad tzv. „oknem“ provádět agregace. Například u každého klienta bych mohl mít informaci, jaký je průměrný věk v jeho segmentu, či u každého zaměstnance informaci o tom, jaký je medián platu na jeho pozici a „bandu“.

Základní syntaxe:

<funkce> (<sloupec / výraz>) OVER (PARTITION BY <skupina> ORDER BY <pořadí>)

Analytické funkce

Základní funkce:

Metaznak	Význam
AVG	Průměrná hodnota.
COUNT	Počet hodnot.
FIRST_VALUE	První hodnota.
LAG	Předcházející hodnota.
LAST_VALUE	Poslední hodnota.
LEAD	Následující hodnota.
LISTAGG	Zřetězení hodnot.
MAX	Maximální hodnota.
MEDIAN	Medián.
MIN	Minimální hodnota.
RANK	Ohodnocení řádků.
ROW_NUMBER	Pořadí řádku.
SUM	Součet.

Analytické funkce

Parametry:

Za sloupcem / výrazem můžou následovat klíčová slova **RESPECT NULLS** nebo **IGNORE NULLS**. Výchozí chování je respektování nullových hodnot.

Partition by:

Definuje skupinu záznamů, nad kterými jsou funkce prováděny. Takže pokud je v klauzuli partitioned by org_key, pak všechny funkce jsou vyhodnocovány pro jednotlivé organizační jednotky. Tedy count by mohl znamenat například počet klientů se stejnou domovskou pobočkou.

Order by:

Pořadí sloupců určuje vyhodnocování hodnot jednotlivých funkcí. Výchozí chování je, že se zpracovávají hodnoty od prvního řádku z výběru až do aktuálního. Tedy například pokud mám pro jednu skupinu tři řádky a použiji funkci sum, pak na prvním řádku je aktuální hodnota, na druhém součet prvního a druhého a nakonec na třetím součet všech tří hodnot.

Toto chování lze změnit pomocí definování okna, které následuje hned po seznamu sloupců v order by.

Analytické funkce

Okno:

ROWS BETWEEN <začátek> **AND** <konec>

<začátek> může nabývat hodnot

UNBOUNDED PRECEDING
CURRENT ROW
Výraz / konstanta **PRECEDING**
Výraz / konstanta **FOLLOWING**

Obdobně <konec>

UNBOUNDED FOLLOWING
CURRENT ROW
Výraz / konstanta **PRECEDING**
Výraz / konstanta **FOLLOWING**

Výchozí chování je **ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**.

Analytické funkce *(23_asql_anafce_01.txt)*

Příklad:

U všech běžných účtů ze Symbols vypište ty měsíce roku 2025, kdy byl zůstatek na účtu vyšší než průměrný zůstatek za všechny předcházející měsíce.

Řešení:

```
select
  acc_key,
  accfc_period
from
  (
    select
      acc_key,
      accfc_period,
      accfc_balance_czk,
      avg(accfc_balance_czk) over (partition by acc_key order by accfc_period) as prumer
    from
      dwh_owner.current_account_facts
    where
      acctp_key = 602
  )
where
  to_char(accfc_period, 'YYYY') = '2025' and
  prumer is not null and
  accfc_balance_czk > prumer;
```

ACC_KEY	ACCFC_PERIOD
40465574	30/06/2025
40465574	31/07/2025
40465574	31/08/2025
40465574	30/09/2025
40465803	31/08/2025
40465803	30/09/2025
40465803	31/10/2025
40465803	30/11/2025

Analytické funkce *(24_asql_anafce_02.txt)*

Příklad:

U všech běžných účtů ze Symbols vypište ty měsíce roku 2025, kdy byl zůstatek na účtu vyšší než průměrný zůstatek za předcházející tři měsíce.

Řešení:

```
select
  acc_key,
  accfc_period
from
  (
    select
      acc_key,
      accfc_period,
      accfc_balance_czk,
      avg(accfc_balance_czk) over (partition by acc_key order by accfc_period rows between 3 preceding and 1 preceding) as prumer
    from
      dwh_owner.current_account_facts
    where
      acctp_key = 602 and
      accfc_period >= to_date('20241031', 'YYYYMMDD')
  )
where
  to_char(accfc_period, 'YYYY') = '2025' and
  prumer is not null and
  accfc_balance_czk > prumer;
```

ACC_KEY	ACCFC_PERIOD
40465574	28/02/2025
40465574	31/03/2025
40465574	30/04/2025
40465574	31/05/2025
40465574	30/06/2025
40465574	31/07/2025
40465574	31/08/2025
40465574	30/09/2025
40465647	31/01/2025

Analytické funkce

Příklad:

Vypište u klienta s pt_key = 3133711, kdy měl jakou domovskou pobočku.

Podívejme se na party_history.

select org_key, pth_valid_from, pth_valid_to from dwh_owner.party_history where pt_key = 3133711 and ptori_key = 4 order by pth_valid_from desc;

ORG_KEY	PTH_VALID_FROM	PTH_VALID_TO
442171	30/10/2024	01/01/3000
442171	03/04/2024	29/10/2024
442171	01/12/2023	02/04/2024
442171	22/04/2023	30/11/2023
442171	01/03/2023	21/04/2023
442171	01/02/2023	28/02/2023
442171	25/01/2023	31/01/2023
442171	04/01/2023	24/01/2023
442171	17/10/2020	03/01/2023
442171	01/09/2020	16/10/2020
442171	15/07/2020	31/08/2020
442171	27/05/2020	14/07/2020
442171	05/12/2019	26/05/2020
442171	02/05/2019	04/12/2019
442171	01/02/2019	01/05/2019
442171	20/07/2018	31/01/2019
442171	01/05/2018	19/07/2018
442171	21/04/2018	30/04/2018
442171	26/01/2018	20/04/2018

ORG_KEY	PTH_VALID_FROM	PTH_VALID_TO
442171	09/09/2017	25/01/2018
442171	08/09/2017	08/09/2017
442171	22/04/2017	07/09/2017
442171	11/03/2017	21/04/2017
442171	10/03/2017	10/03/2017
442171	31/01/2017	09/03/2017
442171	30/01/2017	30/01/2017
6929	30/04/2016	29/01/2017
6929	31/03/2016	29/04/2016
6929	05/02/2016	30/03/2016
6929	04/02/2016	04/02/2016
6929	31/01/2016	03/02/2016
6929	01/07/2015	30/01/2016
6929	30/04/2014	30/06/2015
6929	29/04/2014	29/04/2014
6929	04/02/2012	28/04/2014
6929	01/01/2011	03/02/2012
6929	21/07/2010	31/12/2010
6929	20/07/2010	20/07/2010

ORG_KEY	PTH_VALID_FROM	PTH_VALID_TO
6929	19/07/2010	19/07/2010
6929	08/07/2010	18/07/2010
969	02/04/2010	07/07/2010
969	01/04/2010	01/04/2010
969	22/12/2009	31/03/2010
969	01/10/2009	21/12/2009
969	09/01/2009	30/09/2009
7000	07/01/2009	08/01/2009
7000	21/11/2006	06/01/2009
7000	10/08/2006	20/11/2006
7000	02/03/2006	09/08/2006
7000	23/02/2006	01/03/2006
7000	21/02/2006	22/02/2006
7000	26/05/2005	20/02/2006
1865	31/12/2004	25/05/2005
1865	23/10/2004	30/12/2004
1865	31/01/2004	22/10/2004
1865	17/10/2003	30/01/2004
1865	27/12/2002	16/10/2003

Analytické funkce

Příklad:

Vypište u klienta s pt_key = 3133711, kdy měl jakou domovskou pobočku.

Chceme tedy dosáhnout výsledku:

442171 -> 30.1.2017 – 1.1.3000
6929 -> 8.7.2010 – 29.1.2017
969 -> 9.1.2009 – 7.7.2010
7000 -> 26.5.2005 – 8.1.2009
1865 -> 27.12.2002 – 25.5.2005

Analytické funkce *(25_asql_anafce_03.txt)*

Příklad:

Vypište u klienta s pt_key = 3133711, kdy měl jakou domovskou pobočku.

Nejprve si pro každý záznam dohledáme předcházející a následující hodnotu pobočky.

```
select
  org_key,
  pth_valid_from,
  pth_valid_to,
  lag(org_key) over (order by pth_valid_from) as prev_org_key,
  lead(org_key) over (order by pth_valid_from) as next_org_key
from
  dwh_owner.party_history
where
  pt_key = 3133711 and
  ptori_key = 4;
```

Analytické funkce *(26_asql_anafce_04.txt)*

Příklad:

Vypište u klienta s pt_key = 3133711, kdy měl jakou domovskou pobočku.

Poté najdete datum, kdy došlo ke změně pobočky, tj. org_key <> prev_org_key, resp. org_key <> next_org_key.

```
select
  org_key,
  pth_valid_from,
  pth_valid_to,
  last_value(case when org_key = prev_org_key then null else pth_valid_from end ignore nulls) over (order by pth_valid_from) as new_valid_from,
  last_value(case when org_key = next_org_key then null else pth_valid_to end ignore nulls) over (order by pth_valid_from desc) as new_valid_to
from
  (
    select
      org_key,
      pth_valid_from,
      pth_valid_to,
      lag(org_key) over (order by pth_valid_from) as prev_org_key,
      lead(org_key) over (order by pth_valid_from) as next_org_key
    from
      dwh_owner.party_history
    where
      pt_key = 3133711 and
      ptori_key = 4
    )
order by
  pth_valid_from desc;
```

Analytické funkce *(27_asql_anafce_05.txt)*

Příklad:

Vypište u klienta s pt_key = 3133711, kdy měl jakou domovskou pobočku.

Když provedeme distinct, máme kýžené řešení.

```
Select
  distinct
    org_key,
    last_value(case when org_key = prev_org_key then null else pth_valid_from end ignore nulls) over (order by pth_valid_from) as new_valid_from,
    last_value(case when org_key = next_org_key then null else pth_valid_to end ignore nulls) over (order by pth_valid_from desc) as new_valid_to
from
  (
    select
      org_key,
      pth_valid_from,
      pth_valid_to,
      lag(org_key) over (order by pth_valid_from) as prev_org_key,
      lead(org_key) over (order by pth_valid_from) as next_org_key
    from
      dwh_owner.party_history
    where
      pt_key = 3133711 and
      ptori_key = 4
    )
order by
  new_valid_from desc;
```

Analytické funkce *(28_asql_anafce_06.txt)*

Příklad:

Vypište u klienta s pt_key = 3133711, kdy měl jakou domovskou pobočku.

Představme si ale, že úkolem mohlo být provést deduplikaci, tedy duplicitní řádky bych potřeboval smazat a jeden řádek (např. ten poslední) „prodloužit“. Pak bychom si museli pomoci funkcí row_number.



28_asql_anafce_06.txt

Analytické funkce

Úkol:

U každého člena nějaké ESSO skupiny vypište název této skupiny spolu se všemi jejími členy.

Řešení:

```
select
  p_o.pt_party_name,
  p_s.pt_party_name,
  listagg(p_s.pt_party_name, ';') within group (order by p_s.pt_party_name) over (partition by p_o.pt_key)
from
  dwh_owner.party_relations pr,
  dwh_owner.parties p_o,
  dwh_owner.parties p_s
where
  pr.ptreltp_type_key = 75 and
  sysdate between pr.ptrel_valid_from and pr.ptrel_valid_to and
  pr.pt_object_key = p_o.pt_key and
  pr.ptori_object_key = p_o.ptori_key and
  p_o.ptori_key = 73 and
  pr.pt_subject_key = p_s.pt_key and
  pr.ptori_subject_key = p_s.ptori_key and
  p_s.ptori_key = 4;
```

PT_PARTY_NAME	PT_PARTY_NAME_1	LISTAGG(P_S.PT_PARTY_NAME,';')WITHI
Milan Matějka Group	Matějka Milan	Matějka Milan;Milan Matějka
Milan Matějka Group	Milan Matějka	Matějka Milan;Milan Matějka
Jaroslav Báňa Group	Báňa Jaroslav	Báňa Jaroslav;Jaroslav Báňa
Jaroslav Báňa Group	Jaroslav Báňa	Báňa Jaroslav;Jaroslav Báňa
Daria Podboj Group	MgA. Daria Podboj Havrda	MgA. Daria Podboj Havrda;Podboj Daria
Daria Podboj Group	Podboj Daria	MgA. Daria Podboj Havrda;Podboj Daria
Vladimír Zich Group	ALBION spol. s r.o.	ALBION spol. s r.o.;Zich Vladimír
Vladimír Zich Group	Zich Vladimír	ALBION spol. s r.o.;Zich Vladimír
Simona Pušková Group	Pušková Simona	Pušková Simona;Simona Pušková
Simona Pušková Group	Simona Pušková	Pušková Simona;Simona Pušková
Karel Michna Group	Karel Michna	Karel Michna;Michna Karel

Analytické funkce

Úkol:

Pro všechny běžné účty Symbols spočtěte průměrný zůstatek za leden 2026. Ve dnech, kdy zůstatek neznáme, použije hodnotu z předchozího dne. K ultimo vždy hodnotu máme. Pro nově založené účty použijte 0. Výsledek můžete porovnat na current_account_facts. Pracujte s typem zůstatku accbaltp_key = 735.

Řešení:

```
select
  b.acc_key,
  b.avg_bal,
  c.accfc_mtd_avg_balance_czk
from
  dwh_owner.current_account_facts c,
  (
    select
      acc_key,
      sum(accbal_amount_czk * (next_bal_date - accbal_date)) / 31 as avg_bal
    from
      (
        select
          acc_key,
          decode(accbal_date, to_date('20251231', 'YYYYMMDD'), to_date('20260101', 'YYYYMMDD'), accbal_date) as accbal_date,
          accbal_amount_czk,
          nvl(lead(accbal_date) over (partition by acc_key order by accbal_date), to_date('20260201', 'YYYYMMDD'))  as next_bal_date
        from
          dwh_owner.account_balances
        where
          acctp_key = 602 and
          accbaltp_key = 735 and
          accbal_date between to_date('20251231', 'YYYYMMDD') and to_date('20260131', 'YYYYMMDD')
      )
    group by
      acc_key
  ) b
where
  c.accfc_period = to_date('20260131', 'YYYYMMDD') and
  c.acctp_key = 602 and
  c.acc_key = b.acc_key;
```

ACC_KEY	AVG_BAL	ACCFC_MTD_AVG_BALANCE_CZK
40469041	0	0
40469107	0	0
40469111	10680072.773225806451612903225806451612 9	10680072.773225806451612903225806451612 9
40469113	48204.621290322580645161290322580645161 3	48204.621290322580645161290322580645161 3
40469123	0	0
40469139	0	0
40469152	0	0
40469178	54440.902580645161290322580645161290322 6	54440.902580645161290322580645161290322 6

Dotazy

Máte nějaké dotazy?

Děkuji za pozornost a příště naviděnou!